Database Build Script

Purpose

Allow an evaluator to select a **released version label** (Git tag), retrieve the exact database artifacts from GitHub, create a fresh database, build objects in dependency order, load initial data, validate the result, and stop safely if any step fails.

A release tag identifies a specific repository version. The build must use a tag such as `v1.0.0`, not an unpinned branch such as `main`, so that the same scripts can be reproduced later.

Required repository structure

Use a consistent directory structure. Numbered filenames make the execution order visible and repeatable.

```
project-repository/
├── README.md
├── build/
│   ├── build_database.sh
│   ├── build_database.ps1
│   └── config.example.env
├── database/
│   ├── 00_create_database.sql
│   ├── 01_schema/
│   │   ├── 010_reference_tables.sql
│   │   ├── 020_core_tables.sql
│   │   ├── 030_relationship_tables.sql
│   │   └── 040_constraints.sql
│   ├── 02_programmability/
│   │   ├── 010_views.sql
│   │   ├── 020_functions.sql
│   │   ├── 030_procedures.sql
│   │   └── 040_triggers.sql
│   ├── 03_security/
│   │   ├── 010_roles.sql
│   │   └── 020_grants.sql
│   ├── 04_seed/
│   │   ├── 010_reference_data.sql
│   │   └── 020_sample_data.sql
│   ├── 05_validation/
│   │   ├── 010_object_validation.sql
```

```
│   │   ├── 020_data_validation.sql
│   │   └── 030_smoke_tests.sql
│   └── manifest.txt
└── docs/
    ├── FRD.pdf
    ├── TDD.pdf
    └── ERD.pdf
```

Required repository conventions

- Create a Git tag for every deliverable build, using a consistent pattern such as `v1.0.0`, `v1.1.0`, or `demo-1.0`.
- Build from the selected tag only. Do not use `main` or another changing branch in the demo build.
- Keep each database object or closely related group of objects in a logically named `.sql` file.
- Use numeric prefixes to establish dependency order.
- Create tables before foreign keys when necessary; create views, routines, and triggers only after their referenced tables exist.
- Load reference/lookup data before transactional/sample data.
- Create roles before grants.
- Run validation/smoke-test scripts only after all schema and initial-data scripts succeed.
- Document the required database platform, version, client tools, prerequisites, environment variables, and expected build command in `README.md`.
- Never commit production passwords, API tokens, or database credentials. Use environment variables

High-level build flow

1. **Select the version label**
    - The operator supplies the GitHub owner, repository name, version tag, database host, administrator user, and target database name.
    - Example version label: `v1.0.0`.
2. **Validate prerequisites**
    - Verify required tools are installed: `git`, `mysql`, and optionally `curl`.

- Verify required environment variables are present.
- Verify the database server is reachable.
- Verify the GitHub repository and requested tag exist.

3. **Retrieve the exact tagged source**
   - Clone the repository and check out the requested tag, or download a repository archive using the tag as the reference.
   - Confirm that `git describe --exact-match --tags HEAD` matches the requested tag.
   - Record the tag and resolved commit SHA in the build log.
4. **Validate build contents before changing the database**
   - Verify the expected `database/` directory exists.
   - Verify that the manifest exists and all files listed in it are present.
   - Reject unexpected script order, missing files, or an empty manifest.
   - Optionally validate a checksum for each script listed in the manifest.
5. **Create or reset the target database**
   - In a classroom/demo environment, either create a new uniquely named database or explicitly drop/recreate the target only after confirmation.
   - Do not silently overwrite an existing database.
   - Create a build-log table early so the build outcome and individual script status are traceable.
6. **Build database objects in dependency order**
   - Execute SQL files one at a time in the manifest's ordered sequence.
   - Stop immediately when a script fails.
   - Record each script's start time, end time, success/failure status, SQL/client error output, release tag, and commit SHA.
   - Use `--force` only for scripts intentionally designed to collect validation failures; never use it for the main schema build.
7. **Load initial data**
   - Load controlled reference values first, followed by test/sample data.

- Use explicit transactions inside data scripts when a logical unit requires all-or-nothing behavior.
- Validate row counts and critical lookup values after loading.

8. **Validate the completed build**
   - Verify required tables, keys, indexes, views, procedures, functions, triggers, roles/grants, and seed-data row counts.
   - Run smoke tests that confirm the database is usable.
   - Run selected common business queries and `EXPLAIN` plans to demonstrate expected index/access-path use.
9. **Publish build evidence**
   - Save a timestamped log containing the requested version label, resolved commit SHA, script order, execution results, validation results, and any failures.
   - Include the log and validation output in the repository or demo evidence folder when required by the course.

Manifest example

The manifest is the authoritative build order. It should contain paths relative to the repository root, one per line. Keep comments and blank lines allowed, but ignore them in the build script.

```
# Database build manifest for release v1.0.0
# Execute in this exact order.

database/00_create_database.sql
database/01_schema/010_reference_tables.sql
database/01_schema/020_core_tables.sql
database/01_schema/030_relationship_tables.sql
database/01_schema/040_constraints.sql
database/02_programmability/010_views.sql
database/02_programmability/020_functions.sql
database/02_programmability/030_procedures.sql
database/02_programmability/040_triggers.sql
database/03_security/010_roles.sql
database/03_security/020_grants.sql
database/04_seed/010_reference_data.sql
database/04_seed/020_sample_data.sql
database/05_validation/010_object_validation.sql
```

```
database/05_validation/020_data_validation.sql
database/05_validation/030_smoke_tests.sql
```

Example Bash build script

This example targets **MySQL 8.x** and a Unix-like shell such as macOS, Linux, Git Bash, or WSL. It clones the exact Git tag, verifies it, builds a fresh database from the ordered manifest, captures errors, and writes a build log.

> Before running: create a local configuration file or export the environment variables shown below. Do not place real passwords in the repository.

```
#!/usr/bin/env bash
# build/build_database.sh
# Example usage:
# ./build_database.sh OWNER REPOSITORY v1.0.0 project_demo_db

set -Eeuo pipefail
IFS=$'\n\t'

OWNER="${1:?GitHub owner is required}"
REPO="${2:?GitHub repository name is required}"
VERSION_TAG="${3:?Git tag/version label is required}"
TARGET_DB="${4:?Target database name is required}"

: "${DB_HOST:?Set DB_HOST}"
: "${DB_PORT:=3306}"
: "${DB_ADMIN_USER:?Set DB_ADMIN_USER}"
: "${DB_ADMIN_PASSWORD:?Set DB_ADMIN_PASSWORD}"
: "${DB_APP_USER:=$DB_ADMIN_USER}"
: "${DB_APP_PASSWORD:=$DB_ADMIN_PASSWORD}"
: "${GITHUB_TOKEN:=}"

RUN_ID="$(date -u +%Y%m%dT%H%M%SZ)-${VERSION_TAG}"
WORK_ROOT="${WORK_ROOT:-$(pwd)/.build-work}"
WORK_DIR="${WORK_ROOT}/${REPO}-${RUN_ID}"
REPO_DIR="${WORK_DIR}/repo"
LOG_DIR="${WORK_DIR}/logs"
BUILD_LOG="${LOG_DIR}/build.log"
ERROR_LOG="${LOG_DIR}/error.log"
REPO_URL="https://github.com/${OWNER}/${REPO}.git"

mkdir -p "$LOG_DIR"
```

```
log() {
  printf '%s | %s\n' "$(date -u +'%Y-%m-%dT%H:%M:%SZ')" "$*" | tee -a "$BUILD_LOG"
}

fail() {
  log "BUILD FAILED: $*"
  exit 1
}

on_error() {
  local exit_code="$?"
  local line_no="$1"
  local command="$2"
  printf '%s | ERROR | exit=%s | line=%s | command=%s\n' \
    "$(date -u +'%Y-%m-%dT%H:%M:%SZ')" "$exit_code" "$line_no" "$command" \
    | tee -a "$ERROR_LOG" "$BUILD_LOG" >&2
  exit "$exit_code"
}
trap 'on_error "$LINENO" "$BASH_COMMAND"' ERR

mysql_admin() {
  MYSQL_PWD="$DB_ADMIN_PASSWORD" mysql \
    --protocol=TCP \
    --host="$DB_HOST" \
    --port="$DB_PORT" \
    --user="$DB_ADMIN_USER" \
    --show-warnings \
    "$@"
}

mysql_app() {
  MYSQL_PWD="$DB_APP_PASSWORD" mysql \
    --protocol=TCP \
    --host="$DB_HOST" \
    --port="$DB_PORT" \
    --user="$DB_APP_USER" \
    --database="$TARGET_DB" \
    --show-warnings \
    "$@"
}

require_command() {
  command -v "$1" >/dev/null 2>&1 || fail "Required command not found: $1"
```

```
}

for cmd in git mysql; do
  require_command "$cmd"
done

log "Starting build. owner=${OWNER}; repository=${REPO}; tag=${VERSION_TAG}; database=${TARGET_DB}"
log "Testing database connectivity."
mysql_admin --batch --skip-column-names -e "SELECT VERSION();" >>"$BUILD_LOG" 2>&1 \
  || fail "Cannot connect to MySQL using supplied connection settings."

log "Cloning requested GitHub tag."
if [[ -n "$GITHUB_TOKEN" ]]; then
  REPO_URL="https://x-access-token:${GITHUB_TOKEN}@github.com/${OWNER}/${REPO}.git"
fi

git clone --quiet --no-checkout "$REPO_URL" "$REPO_DIR" >>"$BUILD_LOG" 2>&1 \
  || fail "Git clone failed. Check repository access and GitHub token permissions."

git -C "$REPO_DIR" fetch --quiet --depth 1 origin "refs/tags/${VERSION_TAG}:refs/tags/${VERSION_TAG}" \
  >>"$BUILD_LOG" 2>&1 \
  || fail "Requested tag '${VERSION_TAG}' was not found or could not be fetched."

git -C "$REPO_DIR" checkout --quiet --detach "refs/tags/${VERSION_TAG}" >>"$BUILD_LOG" 2>&1 \
  || fail "Could not check out requested tag '${VERSION_TAG}'."

RESOLVED_TAG="$(git -C "$REPO_DIR" describe --exact-match --tags HEAD 2>/dev/null || true)"
COMMIT_SHA="$(git -C "$REPO_DIR" rev-parse HEAD)"
[[ "$RESOLVED_TAG" == "$VERSION_TAG" ]] \
  || fail "Checked-out source is not exactly tag '${VERSION_TAG}'. Resolved tag: '${RESOLVED_TAG:-none}'."
log "Verified repository release tag=${RESOLVED_TAG}; commit=${COMMIT_SHA}"

MANIFEST="$REPO_DIR/database/manifest.txt"
[[ -f "$MANIFEST" ]] || fail "Missing required build manifest: database/manifest.txt"

mapfile -t SQL_FILES < <(grep -Ev '^\s*(#|$)' "$MANIFEST")
[[ "${#SQL_FILES[@]}" -gt 0 ]] || fail "The build manifest has no executable SQL files."

for relative_file in "${SQL_FILES[@]}"; do
  [[ "$relative_file" == database/*.sql ]] \
    || fail "Manifest entry must be a relative database SQL path: ${relative_file}"
  [[ -f "$REPO_DIR/$relative_file" ]] \
    || fail "Manifest references a missing file: ${relative_file}"
done
```

```
log "Creating build-audit database objects if needed."
mysql_admin <<SQL >>"$BUILD_LOG" 2>&1
CREATE DATABASE IF NOT EXISTS build_admin;
CREATE TABLE IF NOT EXISTS build_admin.build_execution_log (
   build_execution_id BIGINT AUTO_INCREMENT PRIMARY KEY,
   run_id VARCHAR(120) NOT NULL,
   requested_tag VARCHAR(100) NOT NULL,
   commit_sha CHAR(40) NOT NULL,
   target_database VARCHAR(64) NOT NULL,
   script_path VARCHAR(500) NULL,
   status ENUM('STARTED','SUCCEEDED','FAILED') NOT NULL,
   started_at_utc DATETIME NOT NULL,
   completed_at_utc DATETIME NULL,
   error_message TEXT NULL
);
SQL

record_status() {
 local script_path="$1"
 local status="$2"
 local error_message="${3:-}"
 local escaped_error
 escaped_error="${error_message//\\/\\\\}"
 escaped_error="${escaped_error//\'/\'\'}"
 mysql_admin --execute="INSERT INTO build_admin.build_execution_log
  (run_id, requested_tag, commit_sha, target_database, script_path, status, started_at_utc, completed_at_utc, error_message)
  VALUES ('${RUN_ID}', '${VERSION_TAG}', '${COMMIT_SHA}', '${TARGET_DB}', '${script_path}', '${status}',
UTC_TIMESTAMP(),
    CASE WHEN '${status}' IN ('SUCCEEDED','FAILED') THEN UTC_TIMESTAMP() ELSE NULL END,
    NULLIF('${escaped_error}', ''));" >>"$BUILD_LOG" 2>&1 || true
}

log "Creating fresh target database '${TARGET_DB}'."
# Classroom/demo safety: refuse to replace an existing database unless ALLOW_DATABASE_RECREATE=YES.
DB_EXISTS="$(mysql_admin --batch --skip-column-names -e \
 "SELECT COUNT(*) FROM information_schema.schemata WHERE schema_name='${TARGET_DB}';")"
if [[ "$DB_EXISTS" == "1" ]]; then
 [[ "${ALLOW_DATABASE_RECREATE:-NO}" == "YES" ]] \
  || fail "Database '${TARGET_DB}' already exists. Set ALLOW_DATABASE_RECREATE=YES only when replacement is
intentional."
 log "Dropping existing target database because ALLOW_DATABASE_RECREATE=YES."
 mysql_admin --execute="DROP DATABASE \`${TARGET_DB}\`;" >>"$BUILD_LOG" 2>&1 \
  || fail "Could not drop existing database '${TARGET_DB}'."
```

```
fi

mysql_admin --execute="CREATE DATABASE \`${TARGET_DB}\` CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;"
\
 >>"$BUILD_LOG" 2>&1 || fail "Could not create target database '${TARGET_DB}'."

for relative_file in "${SQL_FILES[@]}"; do
 script_file="$REPO_DIR/$relative_file"
 log "Executing ${relative_file}"
 record_status "$relative_file" "STARTED"

 if mysql_app --binary-mode < "$script_file" >>"$BUILD_LOG" 2>>"$ERROR_LOG"; then
  record_status "$relative_file" "SUCCEEDED"
  log "Succeeded: ${relative_file}"
 else
  build_error="SQL script failed: ${relative_file}. See ${ERROR_LOG}."
  record_status "$relative_file" "FAILED" "$build_error"
  fail "$build_error"
 fi
done

log "Build scripts completed. Running final verification."
TABLE_COUNT="$(mysql_admin --batch --skip-column-names -e \
 "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='${TARGET_DB}' AND table_type='BASE
TABLE';")"
INDEX_COUNT="$(mysql_admin --batch --skip-column-names -e \
 "SELECT COUNT(*) FROM information_schema.statistics WHERE table_schema='${TARGET_DB}';")"
TRIGGER_COUNT="$(mysql_admin --batch --skip-column-names -e \
 "SELECT COUNT(*) FROM information_schema.triggers WHERE trigger_schema='${TARGET_DB}';")"

log "Verification summary: tables=${TABLE_COUNT}; indexes=${INDEX_COUNT}; triggers=${TRIGGER_COUNT}"
log "BUILD SUCCEEDED. tag=${VERSION_TAG}; commit=${COMMIT_SHA}; database=${TARGET_DB}; run_id=${RUN_ID}"
printf '\nBuild log: %s\nError log: %s\n' "$BUILD_LOG" "$ERROR_LOG"
```

Example environment setup

Use environment variables or a local configuration file that is excluded from source control.

```
export DB_HOST="127.0.0.1"
export DB_PORT="3306"
export DB_ADMIN_USER="root"
export DB_ADMIN_PASSWORD="replace-with-local-secret"
```

```
# Optional: use a limited application/schema-build account after database creation.
export DB_APP_USER="project_builder"
export DB_APP_PASSWORD="replace-with-local-secret"

# Required only if the repository is private.
export GITHUB_TOKEN="github_pat_example"

# Required only if intentionally replacing an existing classroom/demo database.
export ALLOW_DATABASE_RECREATE="YES"
```

Example invocation:

```
chmod +x build/build_database.sh
build/build_database.sh sanikadeshmukh3 Digital-Document-Signing-Portal v1.0.0 document_portal_demo
```

Example PowerShell build script

For Windows users, the following simplified PowerShell version follows the same approach. Students may submit either a Bash or PowerShell build script unless the project specifies a required platform.

```
# build/build_database.ps1
# Example:
# .\build_database.ps1 -Owner "OWNER" -Repository "REPOSITORY" -VersionTag "v1.0.0" -TargetDatabase
"project_demo_db"

param(
   [Parameter(Mandatory=$true)] [string] $Owner,
   [Parameter(Mandatory=$true)] [string] $Repository,
   [Parameter(Mandatory=$true)] [string] $VersionTag,
   [Parameter(Mandatory=$true)] [string] $TargetDatabase
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

foreach ($variableName in @("DB_HOST", "DB_ADMIN_USER", "DB_ADMIN_PASSWORD")) {
   if ([string]::IsNullOrWhiteSpace([Environment]::GetEnvironmentVariable($variableName))) {
      throw "Required environment variable is missing: $variableName"
   }
}

$DbHost = $env:DB_HOST
```

```
$DbPort = if ($env:DB_PORT) { $env:DB_PORT } else { "3306" }
$DbAdminUser = $env:DB_ADMIN_USER
$DbAdminPassword = $env:DB_ADMIN_PASSWORD
$RunId = "$(Get-Date -Format 'yyyyMMddTHHmmssZ')-$VersionTag"
$WorkDir = Join-Path (Get-Location) ".build-work\$Repository-$RunId"
$RepoDir = Join-Path $WorkDir "repo"
$LogDir = Join-Path $WorkDir "logs"
$BuildLog = Join-Path $LogDir "build.log"
$ErrorLog = Join-Path $LogDir "error.log"

New-Item -ItemType Directory -Force -Path $LogDir | Out-Null

function Write-BuildLog([string] $Message) {
  $line = "$(Get-Date -Format 'yyyy-MM-ddTHH:mm:ssK') | $Message"
  $line | Tee-Object -FilePath $BuildLog -Append
}

function Invoke-MySql([string] $Sql, [string] $Database = "") {
  $arguments = @("--protocol=TCP", "--host=$DbHost", "--port=$DbPort", "--user=$DbAdminUser", "--show-warnings")
  if ($Database) { $arguments += "--database=$Database" }
  $arguments += "--execute=$Sql"
  $env:MYSQL_PWD = $DbAdminPassword
  & mysql @arguments 2>> $ErrorLog
  if ($LASTEXITCODE -ne 0) { throw "MySQL command failed: $Sql" }
}

try {
  foreach ($cmd in @("git", "mysql")) {
    if (-not (Get-Command $cmd -ErrorAction SilentlyContinue)) {
      throw "Required command not found: $cmd"
    }
  }

  Write-BuildLog "Starting build. owner=$Owner; repository=$Repository; tag=$VersionTag; database=$TargetDatabase"
  Invoke-MySql "SELECT VERSION();"

  $repoUrl = "https://github.com/$Owner/$Repository.git"
  if ($env:GITHUB_TOKEN) {
    $repoUrl = "https://x-access-token:$($env:GITHUB_TOKEN)@github.com/$Owner/$Repository.git"
  }

  & git clone --quiet --no-checkout $repoUrl $RepoDir 2>> $ErrorLog
  if ($LASTEXITCODE -ne 0) { throw "Git clone failed." }
```

```
  & git -C $RepoDir fetch --quiet --depth 1 origin "refs/tags/$VersionTag`:refs/tags/$VersionTag" 2>> $ErrorLog
  if ($LASTEXITCODE -ne 0) { throw "Tag '$VersionTag' could not be fetched." }

  & git -C $RepoDir checkout --quiet --detach "refs/tags/$VersionTag" 2>> $ErrorLog
  if ($LASTEXITCODE -ne 0) { throw "Tag '$VersionTag' could not be checked out." }

  $ResolvedTag = (& git -C $RepoDir describe --exact-match --tags HEAD).Trim()
  $CommitSha = (& git -C $RepoDir rev-parse HEAD).Trim()
  if ($ResolvedTag -ne $VersionTag) {
    throw "Checked-out source is not exactly the requested tag. Resolved tag: $ResolvedTag"
  }
  Write-BuildLog "Verified tag=$ResolvedTag; commit=$CommitSha"

  $Manifest = Join-Path $RepoDir "database\manifest.txt"
  if (-not (Test-Path $Manifest)) { throw "Missing database manifest: $Manifest" }

  $SqlFiles = Get-Content $Manifest | Where-Object {
    $_.Trim() -and -not $_.Trim().StartsWith("#")
  }
  if ($SqlFiles.Count -eq 0) { throw "Build manifest does not list SQL scripts." }

  foreach ($relativePath in $SqlFiles) {
    $relativePath = $relativePath.Trim()
    $scriptPath = Join-Path $RepoDir $relativePath
    if (-not (Test-Path $scriptPath)) { throw "Manifest references missing SQL file: $relativePath" }
  }

  $existsSql = "SELECT COUNT(*) FROM information_schema.schemata WHERE schema_name='$TargetDatabase';"
  $env:MYSQL_PWD = $DbAdminPassword
  $dbExists = (& mysql --protocol=TCP "--host=$DbHost" "--port=$DbPort" "--user=$DbAdminUser" --batch --skip-column-
names --execute=$existsSql).Trim()

  if ($dbExists -eq "1") {
    if ($env:ALLOW_DATABASE_RECREATE -ne "YES") {
      throw "Database '$TargetDatabase' exists. Set ALLOW_DATABASE_RECREATE=YES only for intentional replacement."
    }
    Write-BuildLog "Dropping existing database: $TargetDatabase"
    Invoke-MySql "DROP DATABASE ``$TargetDatabase``;"
  }

  Invoke-MySql "CREATE DATABASE ``$TargetDatabase`` CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;"

  foreach ($relativePath in $SqlFiles) {
    $relativePath = $relativePath.Trim()
```

```
    $scriptPath = Join-Path $RepoDir $relativePath
    Write-BuildLog "Executing $relativePath"
    $env:MYSQL_PWD = $DbAdminPassword
    cmd /c "mysql --protocol=TCP --host=$DbHost --port=$DbPort --user=$DbAdminUser --database=$TargetDatabase --
show-warnings < `"$scriptPath`"" 2>> $ErrorLog
    if ($LASTEXITCODE -ne 0) {
      throw "SQL script failed: $relativePath. Review $ErrorLog"
    }
    Write-BuildLog "Succeeded: $relativePath"
  }

  Write-BuildLog "BUILD SUCCEEDED. tag=$VersionTag; commit=$CommitSha; database=$TargetDatabase"
}
catch {
  $message = $_.Exception.Message
  Write-BuildLog "BUILD FAILED: $message"
  Add-Content -Path $ErrorLog -Value "$(Get-Date -Format 'yyyy-MM-ddTHH:mm:ssK') | $message"
  exit 1
}
finally {
  Remove-Item Env:MYSQL_PWD -ErrorAction SilentlyContinue
}
```

SQL exception-handling examples

The external build script stops a failed build. Individual database procedures should also use their own transaction and exception handling so that a failed business operation does not leave partial data behind.

Example: atomic workflow creation procedure

```
DELIMITER //

CREATE PROCEDURE sp_create_workflow(
  IN p_document_version_id BIGINT,
  IN p_owner_user_id BIGINT,
  IN p_workflow_name VARCHAR(200)
)
BEGIN
  DECLARE v_workflow_id BIGINT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
```

```
  INSERT INTO AuditLog
    (ActorUserID, EventType, EventTimestamp, Details)
  VALUES
    (p_owner_user_id, 'WORKFLOW_CREATE_FAILED', UTC_TIMESTAMP(),
     'Workflow creation rolled back because of a database exception.');
  RESIGNAL;
END;

START TRANSACTION;

-- Example authorization and state validation.
IF NOT EXISTS (
  SELECT 1
  FROM DocumentVersion dv
  JOIN Document d ON d.DocumentID = dv.DocumentID
  WHERE dv.DocumentVersionID = p_document_version_id
    AND d.OwnerUserID = p_owner_user_id
    AND dv.VersionStatus = 'CURRENT'
) THEN
  SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Owner is not authorized or the document version is not current.';
END IF;

INSERT INTO Workflow
  (DocumentVersionID, InitiatorUserID, WorkflowName, WorkflowStatus, CreatedAt)
VALUES
  (p_document_version_id, p_owner_user_id, p_workflow_name, 'DRAFT', UTC_TIMESTAMP());

SET v_workflow_id = LAST_INSERT_ID();

-- Insert required steps and assignments here.
-- Each insert is part of the same transaction.

UPDATE Workflow
SET WorkflowStatus = 'ACTIVE', ActivatedAt = UTC_TIMESTAMP()
WHERE WorkflowID = v_workflow_id;

INSERT INTO AuditLog
  (ActorUserID, EventType, EventTimestamp, AffectedEntityType, AffectedEntityID, Details)
VALUES
  (p_owner_user_id, 'WORKFLOW_CREATED', UTC_TIMESTAMP(), 'Workflow', v_workflow_id,
   'Workflow was created and activated.');

COMMIT;
```

```
END//

DELIMITER ;
```

Example: controlled exception handling in a seed-data script

For MySQL scripts, a transaction can be used to make a logical unit of seed data atomic. Use procedures when you need a formal `EXIT HANDLER`; anonymous top-level scripts do not provide the same procedural exception-handler structure.

```
START TRANSACTION;

INSERT INTO Role (RoleCode, RoleName, IsActive)
VALUES
  ('OWNER', 'Document Owner', 1),
  ('REVIEWER', 'Reviewer', 1),
  ('APPROVER', 'Approver', 1),
  ('SIGNER', 'Signer', 1);

-- Any error returned by the mysql client causes the outer build script to stop.
-- The team should design this seed script so it can be safely rerun only if that is an explicit requirement.

COMMIT;
```

Build validation examples

Include platform-specific validation scripts under `database/05_validation/`. The examples below should be adapted to actual object names.

```
-- Verify expected core tables exist.
SELECT table_name
FROM information_schema.tables
WHERE table_schema = DATABASE()
  AND table_type = 'BASE TABLE'
ORDER BY table_name;

-- Verify indexes for a key operational worklist query.
SELECT table_name, index_name, column_name, seq_in_index
FROM information_schema.statistics
WHERE table_schema = DATABASE()
  AND table_name IN ('WorkflowParticipant', 'WorkflowStep', 'Workflow')
ORDER BY table_name, index_name, seq_in_index;
```

```
-- Verify stored procedures and triggers exist.
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = DATABASE()
ORDER BY routine_type, routine_name;

SELECT trigger_name, event_object_table, action_timing, event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = DATABASE()
ORDER BY event_object_table, trigger_name;

-- Verify baseline lookup/sample records loaded.
SELECT 'Role' AS entity_name, COUNT(*) AS row_count FROM Role
UNION ALL
SELECT 'User', COUNT(*) FROM User
UNION ALL
SELECT 'Document', COUNT(*) FROM Document
UNION ALL
SELECT 'Workflow', COUNT(*) FROM Workflow;
```

EXPLAIN demonstration examples

Use a common operational query from the FRD and show its execution plan after the fresh build and initialization. The exact syntax and plan fields depend on the selected database platform.

```
EXPLAIN FORMAT=TREE
SELECT
  wp.WorkflowParticipantID,
  d.Title,
  ws.StepType,
  wp.AssignmentStatus,
  wp.DueDateTime
FROM WorkflowParticipant AS wp
JOIN WorkflowStep AS ws
  ON ws.WorkflowStepID = wp.WorkflowStepID
JOIN Workflow AS w
  ON w.WorkflowID = ws.WorkflowID
JOIN DocumentVersion AS dv
  ON dv.DocumentVersionID = w.DocumentVersionID
JOIN Document AS d
  ON d.DocumentID = dv.DocumentID
WHERE wp.AssignedUserID = 101
```

```
  AND wp.AssignmentStatus = 'ACTIVE'
  AND w.WorkflowStatus = 'ACTIVE'
ORDER BY wp.DueDateTime;
```

During the demo, explain:

- The purpose of the query and which FRD reporting/worklist requirement it supports.
- The predicates used for filtering: for example, assigned user, assignment status, and workflow status.
- Which index is selected or expected, such as `(AssignedUserID, AssignmentStatus, DueDateTime)` on the assignment table.
- Whether the plan shows an index lookup/range scan, a full table scan, a sort, a temporary table, or a join method requiring attention.
- Why the selected index column order supports the query's equality filters and ordering requirements.
- Whether the sample data volume is sufficient to demonstrate meaningful access-path behavior; small data sets may not always cause the optimizer to choose an index even when the index is appropriate.

Required exception-handling behavior

The final solution should demonstrate all of the following:

- **Missing configuration:** Stop before modifying the database if a required argument, environment variable, tool, manifest, or SQL file is missing.
- **GitHub retrieval failure:** Stop if the repository cannot be accessed or if the requested version tag cannot be fetched or checked out.
- **Version verification failure:** Stop if the resolved Git tag does not exactly match the requested version label.
- **Database connectivity failure:** Stop before build execution and report a connection failure clearly.
- **Existing database protection:** Refuse to drop or replace an existing database unless an explicit override is supplied.
- **Schema-script failure:** Stop at the failed file, preserve error output, identify the SQL file that failed, and do not continue to dependent scripts.

- **Transaction failure:** Roll back incomplete multi-row/multi-table business transactions; do not leave partial workflow, approval, assignment, or audit-state changes.
- **Validation failure:** Treat missing required objects, missing seed data, or failed smoke tests as a failed build or failed release candidate.
- **Auditability:** Record build release tag, commit SHA, timestamps, script path, result, and error context in both a file log and, where permitted, a build-log table.
- **Secret protection:** Do not expose tokens or passwords in repository files, console output, source code, or committed logs.